A. Informasi & Ringkasan

1. Identitas

NIM	11S25038
Nama	EVAN BENTA PURBA
Kelas	11IF-2
Judul Praktikum	Ekspresi Kondisional

2. Capaian & Ringkasan Pemahaman

Hasil Capaian setelah mengikuti praktikum:

- A (Penyelesaian): Selesai (1), Tidak Selesai (0).
- B (Pemahaman): Tidak Paham (1), Kurang Paham (2), Cukup Paham (3), Paham (4), Sangat Paham (5).

no	nama	a	b
1	Mempraktikan dasar bahasa pemrograman Haskell	1	4
2	Mengimplementasikan Notasi Fungsional ke Haskell	1	3
3	Menggunakan bahasa pemrograman Haskell untuk menyelesaikan studi kasus tertentu	1	3

3. Foto Praktikan



B. Penilaian Observasi

Setiap Mahasiswa yang mengikuti kegiatan praktikum perlu melakukan penilaian terhadap diri sendiri dan rekan kerja berdasarkan [Indikator Penilaian Observasi Praktikum].

1. Penilaian Terhadap Diri Sendiri

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	4
2	Kemampuan Penyelesaian Tugas	3
3	Kerja Sama Tim	5
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	4

2. Penilaian Terhadap Rekan Kerja

Harus melakukan penilaian terhadap rekan kerja selama aktivitas praktikum berlangsung.

a) Steven manurung (11S25013)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	4
2	Kemampuan Penyelesaian Tugas	3
3	Kerja Sama Tim	5
4	Inisiatif dan Problem Solving	4
5	Kedisiplinan dan Etika Kerja	4

b) Michael siburian (11S25007)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	5
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	5

C. Laporan Aktivitas Praktikum

1. Aktivitas Praktikum 1

1.Dasar haskell

a.Model Data

modelData.hs

```haskell
main :: IO()
main = do
  -- 1) Data Number
  -- mendeklarasikan bilangan bulat
  let dataInt = 42
  -- mendeklarasikan bilangan riil
  let dataDouble = 3.14
  -- menampilkan number
  print dataInt
  print dataDouble

  -- 2) Data Character
  -- mendeklarasikan character
  let dataChar = 'A'
  -- menampilkan character
  print dataChar

  -- 3) Data String
  -- mendeklarasikan string
  let dataStr = "Belajar bahasa pemrograman Haskell bersama"
  let dataStr2 = "Abdullah Ubaid"
  -- menampilkan string
  putStrLn dataStr
  putStrLn dataStr2

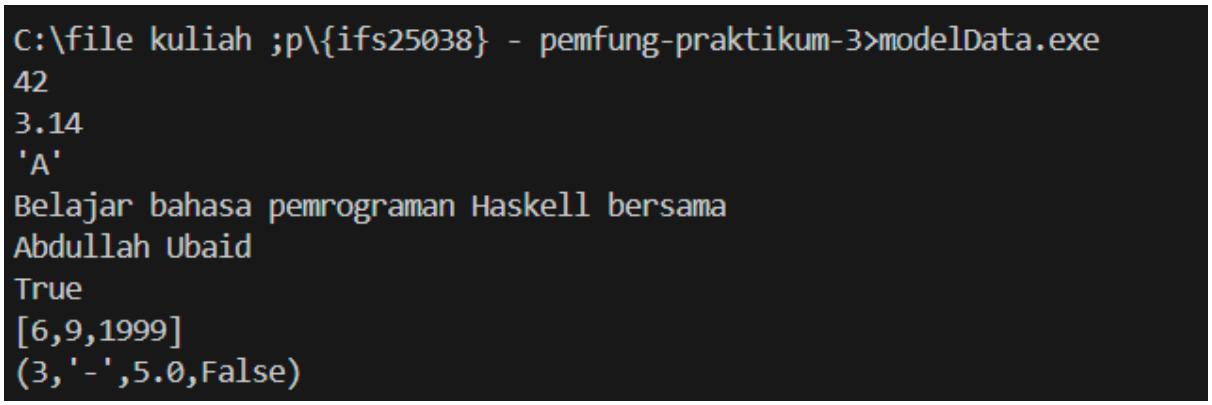
  -- 3) Data Boolean
  -- mendeklarasikan boolean
  let dataBool = True
  -- menampilkan boolean
  print dataBool

  -- 3) Data List
  -- mendeklarasikan list
  let myList = [6, 9, 1999]
  -- menampilkan list
  print myList

  -- 4) Data Tuple
  -- mendeklarasikan list
  let myTuple = (3, '-', 5.0, False)
  -- menampilkan tuple
  print myTuple
```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc modelData.hs
[1 of 2] Compiling Main             ( modelData.hs, modelData.o )
[2 of 2] Linking modelData.exe [Objects changed]

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>modelData.exe
42
3.14
'A'
Belajar bahasa pemrograman Haskell bersama
Abdullah Ubaid
True
[6,9,1999]
(3,'-',5.0,False)
```



Analisis :

"let" merupakan perintah dal variabeam Haskell untuk menyatakan sebuah variable. Contoh :

let dataInt = 42

maka dataInt merupakan sebuah variable dengan nilai 42

Model data (tipe data) :

- Integer(Int)

Integer merupakan tipe data bilangan bulat yang ditulis pada program Haskell sebagai **Int**

Contoh : 1, -1, 100, 2, dsb

- Double

Double merupakan tipe data bilangan rill yang ditulis pada program Haskell sebagai **Double**

Contoh : 2.00, 2.12, 3.14, 0.000001, dsb

- Character(Char)

Character merupakan tipe data satu huruf atau satu karakter. Tipe data ini ditandai dengan petik satu (') dan ditulis **Char**

Contoh : 'a', 'b', 'z'

- String

String merupakan tipe data yang berbentuk teks. Tipe data ini ditandai dengan petik dua (") dan ditulis **String**

- Boolean

Boolean merupakan tipe data yang hanya berisi **True** atau **False.**

Tipe Daftar :

- List

List merupakan tipe daftar sederhana yang hanya dapat memuat satu tipe data. List ditandai dengan tanda kurung siku ([).

Contoh : [1, 2, 3]

- Tuple

Tuple merupakan tipe daftar yang memuat lebih dari satu tipe data. Tuple ditandai dengan tanda kurung biasa (()

b.operator Aritmatika

operatorAritmatika.hs

```haskell
main :: IO ()
main = do
  let
    nilaiInt :: Int
    nilaiInt = 20
  let
    nilaiInt2 :: Int
    nilaiInt2 = 11
  let
    nilaiDouble :: Double
    nilaiDouble = 20.0
  let
    nilaiDouble2 :: Double
    nilaiDouble2 = 11.0

  -- melakukan penjumlahan
  let hasilPenjumlahan = nilaiInt + nilaiInt2
  let hasilPenjumlahan2 = nilaiDouble + nilaiDouble2
  print hasilPenjumlahan
  print hasilPenjumlahan2

  -- melakukan pengurangan
  let hasilPengurangan = nilaiInt - nilaiInt2
  let hasilPengurangan2 = nilaiDouble - nilaiDouble2
  print hasilPengurangan
  print hasilPengurangan2

  -- melakukan perkalian
  let hasilPerkalian = nilaiInt * nilaiInt2
  let hasilPerkalian2 = nilaiDouble * nilaiDouble2
  print hasilPerkalian
  print hasilPerkalian2

  -- melakukan pembagian
  let hasilPembagian = fromIntegral nilaiInt / fromIntegral nilaiInt2
  let hasilPembagian2 = nilaiDouble / nilaiDouble2
  print hasilPembagian
  print hasilPembagian2

  -- mengambil hasil sisa bagi
  let hasilSisaBagi = nilaiInt `mod` nilaiInt2
  let hasilSisaBagi2 = floor nilaiDouble `mod` floor nilaiDouble2
```

```haskell
-- melakukan pembagian
let hasilPembagian = fromIntegral nilaiInt / fromIntegral nilaiInt2
let hasilPembagian2 = nilaiDouble / nilaiDouble2
print hasilPembagian
print hasilPembagian2

-- mengambil hasil sisa bagi
let hasilSisaBagi = nilaiInt `mod` nilaiInt2
let hasilSisaBagi2 = floor nilaiDouble `mod` floor nilaiDouble2
print hasilSisaBagi
print hasilSisaBagi2

-- melakukan pemangkatan
let hasilPangkat = nilaiInt ^ nilaiInt2
let hasilPangkat2 = nilaiDouble ** nilaiDouble2
print hasilPangkat
print hasilPangkat2

-- mengambil hasil akar
let hasilAkar = sqrt (fromIntegral nilaiInt)
let hasilAkar2 = sqrt nilaiDouble2
print hasilAkar
print hasilAkar2
```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc operatorAritmatika.hs
[1 of 2] Compiling Main             ( operatorAritmatika.hs, operatorAritmatika.o )
31
31
31.0
9
9.0
220
220.0
1.8181818181818181
1.8181818181818181
9
9
204800000000000
9
204800000000000
2.048e14
2.048e14
4.47213595499958
4.47213595499958
3.3166247903554
3.3166247903554
```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>operatorAritmatika.exe
31
31.0
9
9.0
220
220.0
1.8181818181818181
1.8181818181818181
9
9
204800000000000
2.048e14
4.47213595499958
3.3166247903554
```

Analisis :
Operasi aritmatika dapat dilambangkan sebagai berikut :

- Penjumlahan(+)
  Contoh : 1 + 1 = 2
- Pengurangan(-)
  Contoh : 1 - 1 = 0
- Perkalian(*)
  Contoh : 1 * 1 = 1
- Pembagian(/)
  Pada kasus pembagian, bilangan yang akan dimunculkan harus bilangan rill. Maka **fromIntegral** diperlukan untuk mengubah tipe Integer menjadi Double.
  Contoh : 1.00 / 1.00 = 1.00
- Sisa Bagi('mod')
  Contoh : 7 'mod' 3 = 1
- Perpangkatan(^ atau **)
  Contoh : 3 ^ 3 = 27
  3 ** 3 = 27
- Akar kuadrat(sqrt())
  Pada kasus aka kuadrat, bilangan yang akan dimunculkan harus bilangan rill. Maka **fromIntegral** diperlukan
  Contoh : sqrt(fromIntegral 9) = 3.00

c.operator Perbandingan

operatorPerbandingan.hs

```haskell
main :: IO()
main = do
  let nilaiInt :: Int
      nilaiInt = 20
  let nilaiInt2 :: Int
      nilaiInt2 = 20

  -- mengecek kesamaan data
  let hasil = nilaiInt == nilaiInt2
  print hasil

  -- mengecek ketidak samaan data
  let hasil = nilaiInt /= nilaiInt2
  print hasil

  -- mengecek apakah data lebih besar
  let hasil = nilaiInt > nilaiInt2
  print hasil

  -- mengecek apakah data lebih besar dan sama dengan
  let hasil = nilaiInt >= nilaiInt2
  print hasil

  -- mengecek apakah data lebih kecil
  let hasil = nilaiInt < nilaiInt2
  print hasil

  -- mengecek apakah data lebih kecil dan sama dengan
  let hasil = nilaiInt <= nilaiInt2
  print hasil
```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc operatorPerbandingan.hs
[2 of 2] Linking operatorPerbandingan.exe

True
True
False
False
True
False
True
```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>operatorPerbandingan.exe
True
False
False
True
False
True
```

Analisis :
Pada matematika ada namanya persamaan dan pertaksamaan. Dalam haskell memuat persamaan dan pertaksamaan tersebut, seperti berikut :

- Sama dengan (==)
- Tidak sama dengan (/=)

- Lebih besar (>)
- Lebih kecil (<)
- Lebih besar atau sama dengan (>=)
- Lebih kecil atau sama dengan (<=)

d.operator Logika

operatorLogika.hs

```haskell
main :: IO()
main = do
  let nilai :: Bool
      nilai = True
  let nilai2 :: Bool
      nilai2 = False
  let nilai3 :: Bool
      nilai3 = True
  let nilai4 :: Bool
      nilai4 = False

  -- operator logika and
  let result = nilai && nilai2
  let result2 = nilai && nilai3
  let result3 = nilai2 && nilai4
  print result
  print result2
  print result3

  -- operator logika or
  let result = nilai || nilai2
  let result2 = nilai || nilai3
  let result3 = nilai2 || nilai4
  print result
  print result2
  print result3

  -- operator logika not
  let result = not nilai
  let result2 = not nilai2
  print result
  print result2
```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc operatorLogika.hs
[1 of 2] Compiling Main             ( operatorLogika.hs, operatorLogika.o )
[2 of 2] Linking operatorLogika.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>operatorLogika.exe
False
True
False
True
True
False
False
True
```

Analisis :
Boolean memuat nilai **True** dan **False** saja. Pada mata kuliah matematika diskrit, dipelajari tabel kebenaran **dan** & **atau.**

Haskell memiliki perintah untulk menentukan nilai kebenaran. Contohnya : p dan q & p atau q. Dengan lambang **dan** sebagai **&&** lalu **atau** dengan lambang **||**.

Konjungsi

p	q	p ∧ q
B	B	B
B	S	S
S	B	S
S	S	S

Disjungsi

p	q	p ∨ q
B	B	B
B	S	B
S	B	B
S	S	S

e.decision making

```
decisionMaking.hs
1   main :: IO()
2   main = do
3     let nilai :: Bool
4         nilai = True
5     let nilai2 :: Bool
6         nilai2 = False
7     let nilai3 :: Bool
8         nilai3 = True
9     let nilai4 :: Bool
10        nilai4 = False
11
12    -- if-else statements
13    if nilai && nilai3
14      then putStrLn "Memilih Jalur 1a"
15      else putStrLn "Memilih Jalur 1b"
16
17    -- if-elseif-else statements
18    if nilai && not nilai3
19      then putStrLn "Memilih Jalur 2a"
20    else if nilai && nilai3
21      then putStrLn "Memilih Jalur 2b"
22    else putStrLn "Memilih Jalur 2c"
```

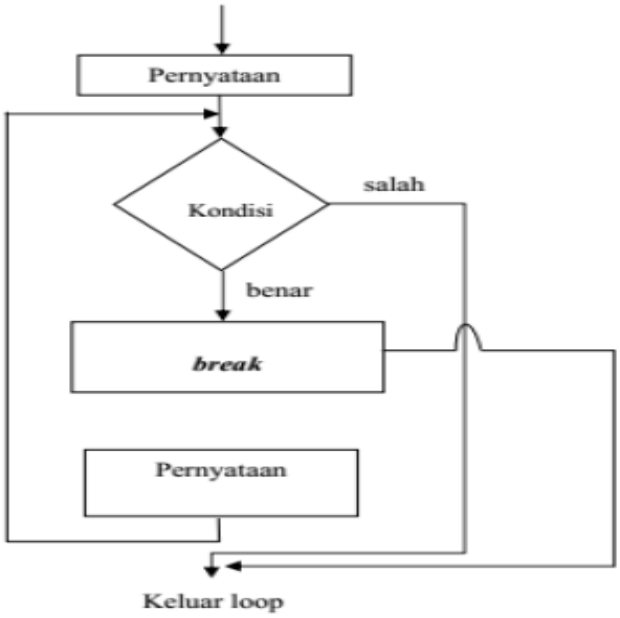
```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc decisionmaking.hs
[1 of 2] Compiling Main             ( decisionmaking.hs, decisionmaking.o )
[2 of 2] Linking decisionmaking.exe [Objects changed]

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>decisionMaking.exe
Memilih Jalur 1a
Memilih Jalur 2b

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>
```

Analisis :
Dalam pemrograman, if-else digunakan untuk membuat keputusan berdasarkan kondisi tertentu. Ini adalah salah satu struktur kontrol alur yang paling umum. if-else itu percabangan kondisi. Jadi, ada banyak kemungkinan yang dapat terjadi. Bila kondisi pertama benar, maka apa yang diminta akan dimunculkan. Bila kondisi pertama salah, maka computer akan mengecek pada kondisi kedua yang kita buat. Jika kondisi kedua benar, maka apa yang kita

perintahkan akan dimunculkan. Jika sebaliknya, maka akan mengecek ke kondisi ketiga dan seterusnya.
Jika seluruh kondisi salah, maka akan adanya perintah else. else berguna untuk menjadi alternatif bila semua kondisinya salah dan akan mengeluarkan output yang diperintahkan.

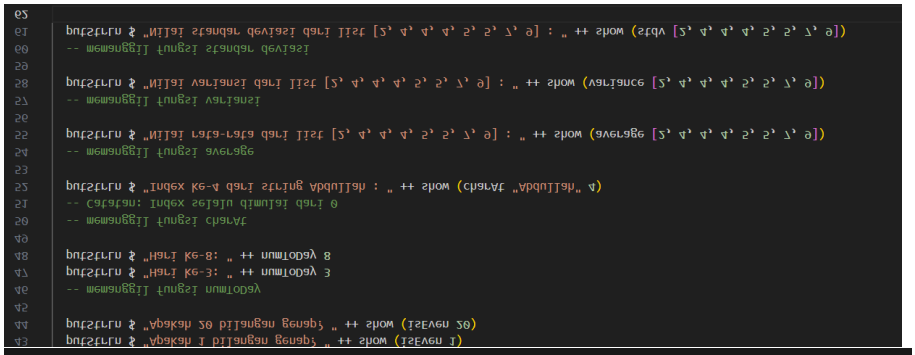


f.fungction

function.hs

```haskell
fungsi  untuk memeriksa apakah suatu bilangan itu genap atau ganjil -- fungsi ini menerima 1 bilangan bulat dan mengembalikan data Boolean
en :: Int -> Bool
en x = even x
 fungsi untuk mengubah bilangan bulat menjadi nama hari -- fungsi ini menerima 1 bilangan bulat dan mengembalikan data String
oDay :: Int -> String
oDay n = case n of
 -> "Senin"
 -> "Selasa"
 -> "Rabu"
 -> "Kamis"
 -> "Jumat"
 -> "Sabtu"
 -> "Minggu"
 -> "Tidak valid"
fungsi untuk mengambil karakter pada string di posisi tertentu -- fungsi ini menerima 1 String dan 1 bilangan bulat dan mengembalikan data Ch
At :: String -> Int -> Char
At str position -- 2 parameter (String, Int)
 position >= 0 && position < length str = str !! position
 otherwise = error "Posisi tidak valid"
fungsi untuk menghitung rata-rata pada daftar bilangan bulat -- fungsi ini menerima daftar bilangan bulat dan mengembalikan data Double
age :: [Int] -> Double
age listInt
 null listInt = 0.0  -- Mengatasi kasus daftar kosong untuk menghindari pembagian nol
 otherwise = fromIntegral (sum listInt) / fromIntegral (length listInt)
fungsi untuk menghitung varians pada daftar bilangan bulat -- fungsi ini menerima daftar bilangan bulat dan mengembalikan data Double
iance :: [Int] -> Double
iance listInt
 length listInt <= 1 = 0.0  -- Handle empty or single-element list to avoid division by zero
 otherwise = sumSquaredDiff / fromIntegral (length listInt - 1)  -- Use sample variance (divide by n-1)
ere
 mean = average listInt
 squaredDiff = [(fromIntegral x - mean) ^ 2 | x <- listInt]
 sumSquaredDiff = sum squaredDiff
fungsi untuk menghitung standar deviasi pada daftar bilangan bulat -- fungsi ini menerima daftar bilangan bulat dan mengembalikan data Double
 :: [Int] -> Double
 listInt
 length listInt <= 1 = 0.0  -- Mengatasi kasus daftar kosong atau daftar dengan satu elemen
 otherwise = sqrt (variance listInt)

 :: IO()
 = do
 memanggil fungsi isEven
```



```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc function.hs
[1 of 2] Compiling Main             ( function.hs, function.o )
[2 of 2] Linking function.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>function.exe
Apakah 1 bilangan genap? False
Apakah 20 bilangan genap? True
Hari ke-3: Rabu
Hari ke-8: Tidak valid
Index ke-4 dari string Abdullah : 'l'
Nilai rata-rata dari list [2, 4, 4, 4, 5, 5, 7, 9] : 5.0
Nilai variansi dari list [2, 4, 4, 4, 5, 5, 7, 9] : 4.571428571428571
Nilai standar deviasi dari list [2, 4, 4, 4, 5, 5, 7, 9] : 2.138089935299395
```

Analisis :
Fungsi didefinisikan dengan cara memberikan nama fungsi, parameter, dan ekspresi hasil.
Contoh fungsi sederhana :

```
tambahDua :: Int -> Int
tambahDua x = x + 2
```

Pemanggilan fungsi dalam Haskell dilakukan dengan menyebutkan nama fungsi diikuti oleh argumen tanpa menggunakan tanda kurung.
Contoh :

```
tambahDua 5
```

maka nilai yang akan dimunculkan adalah 7

```
tambahDua x = x + 2
tambahDua 5 = 5 + 2
            = 7
```

2.mengimplementasikan notasi fungsional ke haskell
a)bilangan positif

BILANGAN POSITIF	isPositif(x)
DEFENISI DAN SEPESIFIKASI isPositif : integer → boolean { isPositif(x) memeriksa apakah sebuah bilangan integer itu positif atau tidak }	
REALISASI isPositif(x) : x >= 0	

isPositif.hs

```haskell
-- isPositif(x) memeriksa apakah sebuah bilangan integer itu
-- positif atau tidak
isPositif :: Int -> Bool
isPositif x = x >= 0

main :: IO()
main = do
  -- memanggil fungsi isPositif
  putStrLn $ "Apakah -10 positif? " ++ show (isPositif (-10))
  putStrLn $ "Apakah -1 positif? " ++ show (isPositif (-1))
  putStrLn $ "Apakah 0 positif? " ++ show (isPositif 0)
  putStrLn $ "Apakah 1 positif? " ++ show (isPositif 1)
  putStrLn $ "Apakah 10 positif? " ++ show (isPositif 10)

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc isPositif.hs
[1 of 2] Compiling Main             ( isPositif.hs, isPositif.o )
[2 of 2] Linking isPositif.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>isPositif.exe
Apakah -10 positif? False
Apakah -1 positif? False
Apakah 0 positif? True
Apakah 1 positif? True
Apakah 10 positif? True

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>
```

Analisis :

1. Dibuatlah fungsi ”isPositif” yang menerima bilangan bulat dan akan menghasilkan nilai boolean
2. Lalu fungsi tersebut berguna untuk mengecek kebenaran apakah **True** atau **False** bilangan yang dimasukkan tersebut lebih besar atau sama dengan 0.

Contoh :
x > 0 -> True
x < 0 -> False
x = 3
3 > 0, maka akan ditampilkan nilai **True.**
x = -1
-1 < 0, maka akan ditampilkan nilai **False.**

3. Fungsi akan dipanggil dengan cara
   putStrLn $ "Apakah 3 positif?" **++ show isPositif 3**
   maka akan muncul Apakah 3 positif? True

b) nilaiValid

NILAI VALID	isValid(x)
DEFENISI DAN SEPESIFIKASI isValid : integer → boolean { isValid(x) memeriksa apakah sebuah bilangan integer (x) valid atau tidak. Valid jika bilangan x berada pada rentang nilai 0 sampai 100, bilangan lainnya dianggap tidak valid. }	
REALISASI isValid(x) : x >= 0 and x <= 100	

isValid.hs

```haskell
-- isValid(x) memeriksa apakah sebuah bilangan integer (x)
-- valid atau tidak. Valid jika bilangan x berada pada rentang nilai
-- sampai 100, bilangan lainnya dianggap tidak valid.
isValid :: Int -> Bool
isValid x = x >= 0 && x <= 100

main :: IO()
main = do
  -- memanggil fungsi isValid
  putStrLn $ "Apakah -10 valid? " ++ show (isValid (-10))
  putStrLn $ "Apakah -1 valid? " ++ show (isValid (-1))
  putStrLn $ "Apakah 0 valid? " ++ show (isValid 0)
  putStrLn $ "Apakah 1 valid? " ++ show (isValid 1)
  putStrLn $ "Apakah 10 valid? " ++ show (isValid 10)
  putStrLn $ "Apakah 100 valid? " ++ show (isValid 100)
  putStrLn $ "Apakah 101 valid? " ++ show (isValid 101)
  putStrLn $ "Apakah 111 valid? " ++ show (isValid 111)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc isValid.hs
[1 of 2] Compiling Main             ( isValid.hs, isValid.o )
[2 of 2] Linking isValid.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>isValid.exe
Apakah -10 valid? False
Apakah -1 valid? False
Apakah 0 valid? True
Apakah 1 valid? True
Apakah 10 valid? True
Apakah 100 valid? True
Apakah 101 valid? False
Apakah 111 valid? False

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>
```

Analisis :

1. Dibuatlah fungsi ”isValid” yang menerima bilangan bulat dan akan menghasilkan nilai boolean

2. Lalu fungsi tersebut berguna untuk mengecek kebenaran apakah True atau False bilangan yang dimasukkan tersebut lebih besar atau sama dengan 0.

Contoh :
x >= 0 && x <= 100 -> True
selain dari situ -> False
x = 3
3 > 0, maka akan ditampilkan nilai True.
x = -1
-1 < 0, maka akan ditampilkan nilai False.

3. Fungsi akan dipanggil dengan cara
putStrLn $ "Apakah 3 valid?" ++ show isValid 3
maka akan muncul Apakah 3 valid? True

c) least Square

NILAI VALID	ls(x1, y1, x2, y2)
DEFENISI DAN SEPESIFIKASI ls : 4 integer → real { ls(x1, y1, x2, y2) menghasilkan akar dari penjumlahan pangkat dua dari hasil pengurangan y2 dengan y1 dan pangkat dua dari hasil pengurangan x2 dengan x1. }	
FUNGSI ANTARA c : integer → integer { c(x) menghasilkan pangkat dua dari x } cd : 2 integer → integer { cd(x,y) menghasilkan pangkat dua dari hasil pengurangan x dengan y }	
REALISASI c(x) = x * x cd(x, y) = c(x-y)	

```
ls(x1, y1, x2, y2) = let cd1 = cd(y2, y1),
                         cd2 = cd(x2,x1),
                         z   = cd1 + cd2
                     in  sqrt (z)
```

ls.hs

```haskell
c :: Integer -> Integer
c x = x * x

cd :: Integer -> Integer -> Integer
cd x y = c (x - y)

ls :: Integer -> Integer -> Integer -> Integer -> Double
ls x1 y1 x2 y2 =
    let cd1 = cd y2 y1
        cd2 = cd x2 x1
        z = cd1 + cd2
    in sqrt (fromIntegral z)

main :: IO ()
main = do
    putStrLn "Masukkan x1:"
    inputX1 <- getLine
    putStrLn "Masukkan y1:"
    inputY1 <- getLine
    putStrLn "Masukkan x2:"
    inputX2 <- getLine
    putStrLn "Masukkan y2:"
    inputY2 <- getLine

    let parsedX1 = reads inputX1 :: [(Integer, String)]
        parsedY1 = reads inputY1 :: [(Integer, String)]
        parsedX2 = reads inputX2 :: [(Integer, String)]
        parsedY2 = reads inputY2 :: [(Integer, String)]

    if null parsedX1 || null parsedY1 || null parsedX2 || null parsedY2
        then putStrLn "Input tidak valid! Masukkan bilangan bulat."
        else do
            let (x1, _) = head parsedX1
                (y1, _) = head parsedY1
                (x2, _) = head parsedX2
                (y2, _) = head parsedY2
                hasil = ls x1 y1 x2 y2
            putStrLn ("Hasil Least Square: " ++ show hasil)

```

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ls.exe
Masukkan x1:
9
Masukkan y1:
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ls.exe
Masukkan x1:
9
Masukkan y1:
Masukkan x1:
9
Masukkan y1:
Masukkan y1:
4
Masukkan x2:
7
Masukkan y2:
6
Hasil Least Square: 2.8284271247461903
Masukkan x2:
7
Masukkan y2:
6
Hasil Least Square: 2.8284271247461903

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>
```

Analisis :

1. Dibuat fungsi dengan nama ls yang menerima 4 bilangan bulat, yaitu : x1, x2, y1, dan y2.
2. Dibuat fungsi antara c yang menerima 1 bilangan bulat x
3. Dibuat fungsi antara cd yang menerima 2 bilangan bulat x dan y
4. Pada fungsi antara c, bilangan bulat x akan dikuadratkan
5. Pada fungsi antara cd, bilangan bulat x dan y akan dioperasikan bersama dengan fungsi c. Fungsi cd akan melakukan operasi perkalian kuadrat menggunakan fungsi c dengan hasil dari selisih antara x dan y
6. Pada fungsi ls, akan dibuat 3 variabel.
   - Variabel pertama adalah cd1 yang merupakan hasil operasi dari fungsi cd dengan bilangan y2 sebagai x dan y1 sebagai y.
   - Variabel kedua adalah cd2 yang merupakan hasil operasi fungsi cd dengan bilangan x2 sebagai x dan x1 sebagai y.
   - Variabel ketiga adalah z yang merupakan jumlah dari cd1 dengan cd2.
7. Fungsi ls akan menggunakan hasil dari fungsi z untuk menghitung akar kuadrat dari hasil fungsi z.

d) nilai Absolut

NILAI ABSOLUTE	mathABS(x)
DEFENISI DAN SEPESIFIKASI mathABS : integer → integer { mathABS(x) menentukan nilai absolute dari x }	
REALISASI mathABS(x) : depend on x x > 0 : x x = 0 : 0 else : -(x)	

mathABS.hs

```haskell
mathABS :: Integer -> Integer
mathABS x
    | x > 0     = x
    | x == 0    = 0
    | otherwise = -x

main :: IO ()
main = do
    putStrLn "Masukkan sebuah bilangan integer:"
    input <- getLine
    let parsed = reads input :: [(Integer, String)]
    if null parsed
        then putStrLn "Input tidak valid! Harus bilangan bulat."
        else do
            let (x, _) = head parsed
            putStrLn ("Nilai absolutnya adalah: " ++ show (mathABS x))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc mathABS.hs
[1 of 2] Compiling Main             ( mathABS.hs, mathABS.o )
[2 of 2] Linking mathABS.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>mathABS.exe
Masukkan sebuah bilangan integer:
9
Nilai absolutnya adalah: 9
```

Analisis :

1) Diberikan fungsi bernama mathABS yang menerima bilangan bulat x.
2) Bilangan x akan diperiksa apakah lebih sama dengan nol atau lebih kecil nol(bernilai negatif).

3) Bila bilangan x lebih besar sama dengan nol, maka akan menghasilkan bilangan x juga.
4) Jika bilangan x lebih kecil dari nol(negatif), maka bilangan x akan dikalikan dengan negatif untuk menghasilkan bilangan x yang positif.

e)Nilai MAX

Nilai Max2	max2(x, y)
DEFENISI DAN SEPESIFIKASI max2 : 2 integer → integer { max2(x, y) mengambil nilai terbsesar dari 2 bilangan integer }	
REALISASI max2(x, y) : if x > y then x else y	

max2.hs

```haskell
max2 :: Integer -> Integer -> Integer
max2 x y =
    if x > y    Hlint Suggestion: Use max. Replace with: max x y
        then x
        else y


main :: IO ()
main = do
    putStrLn "Masukkan bilangan pertama:"
    input1 <- getLine
    putStrLn "Masukkan bilangan kedua:"
    input2 <- getLine

    let parsed1 = reads input1 :: [(Integer, String)]
        parsed2 = reads input2 :: [(Integer, String)]

    if null parsed1 || null parsed2
        then putStrLn "Input tidak valid! Harus bilangan bulat."
        else do
            let (x, _) = head parsed1
                (y, _) = head parsed2
                hasil = max2 x y
            putStrLn ("Bilangan terbesar adalah: " ++ show hasil)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[2 of 2] Linking max2.exe

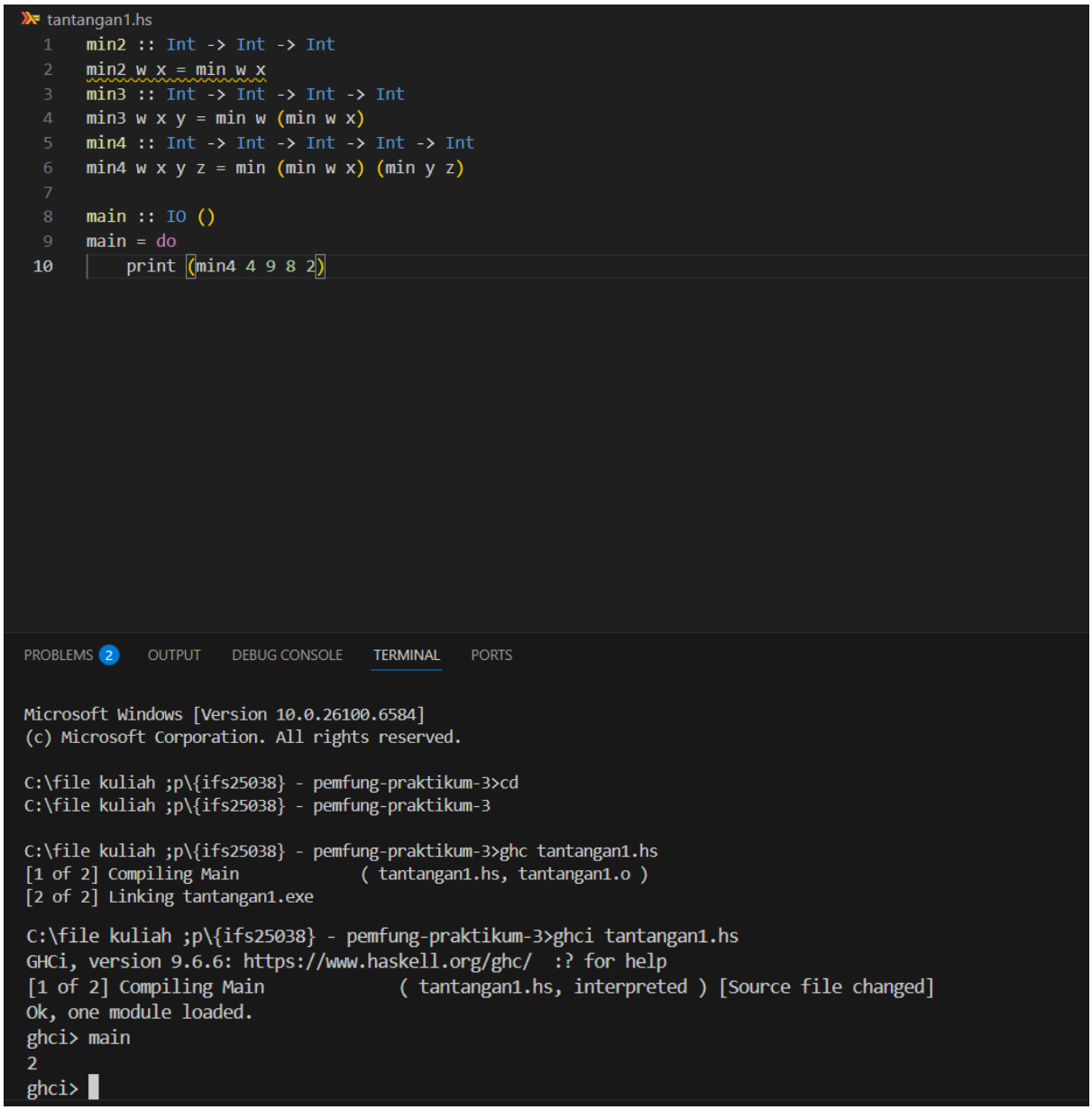
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>max2.exe
Masukkan bilangan pertama:
5
Masukkan bilangan kedua:
4
Bilangan terbesar adalah: 5

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>
```

Analisis :
1) Diberikan fungsi max2 dengan menerima 2 bilangan bulat x dan y.
2) Bilangan x dan y akan diperiksa. Jika x lebih besar dari y, maka nilai x akan dimunculkan.
3) Jika terjadi sebaliknya dan nilai x sama dengan y, maka nilai y akan dimunculkan.

3.Studi kasus
a.nilai min4

min4(v, x, y, z) adalah fungsi untuk menentukan bilangan bulat terkecil dari v, x, y dan z. Gunakan fungsi antara min2 dan min3, yang mirip dengan fungsi max2.



b.Wujud Zat Air

WujudZatAir(x) adalah fungsi untuk melakukan pengecekan wujud zat dari air, berdasarkan suhu air x, dengan x (bilangan real) yang merupakan masukan ke fungsi. Jika suhu di bawah 0◦C maka fungsi mengembalikan teks PADAT, jika suhu lebih besar atau sama dengan 0◦C namun lebih kecil dari 100◦C maka fungsi mengembalikan teks CAIR, jika suhu lebih besar atau sama dengan 100◦C maka fungsi mengembalikan teks GAS.

tantangan2.hs

```haskell
--menentukan wujud zat cair berdasarkan suhu
suhu :: Double -> String
suhu x
  | x <= 0    = "Padat"
  | x >= 100  = "Gas"
  | otherwise = "Cair"

main :: IO ()
main = do
  x <- readLn :: IO Double
  print (suhu x)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
`clang.exe' failed in phase `Linker'. (Exit code: 1)

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc tantangan2.hs
[1 of 2] Compiling Main             ( tantangan2.hs, tantangan2.o )
[2 of 2] Linking tantangan2.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>tantangan2.exe
ghci tantangan2.hs
tantangan2.exe: user error (Prelude.readIO: no parse)

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghci tantangan2.hs
GHCi, version 9.6.6: https://www.haskell.org/ghc/  :? for help
Ok, one module loaded.
ghci> main
90
"Cair"
ghci>
```

a) c Grade Nilai

grade(x) adalah fungsi untuk menentukan bilangan grade yang diperoleh oleh seorang mahasiswa untuk nilai ujian x (bilangan real). Dengan ketentuan seperti berikut:

Nilai (x)	Grade
x ≥ 80	A
72 ≤ x < 80	AB
65 ≤ x < 72	B
57 ≤ x ≤ 65	BC
50 ≤ x < 57	C
35 ≤ x < 50	D
x < 35	E

tantangan3.hs

```haskell
--menentukan grade dan skala nilai
nilai :: Double -> String
nilai x
  | x >= 79.5 && x <= 100 = "Grade A dan Skala 4"
  | x >= 72 && x < 79.5  = "Grade AB dan Skala 3.5"
  | x >= 64.5 && x < 72  = "Grade B dan Skala 3"
  | x >= 57 && x < 64.5  = "Grade BC dan Skala 2.5"
  | x >= 49.5 && x < 57  = "Grade C dan Skala 2"
  | x >= 34 && x < 49.5  = "Grade D dan Skala 1"
  | x >= 0 && x < 34     = "Grade E dan Skala 0"
  | otherwise          = "Nilai tidak terdefinisi"

main :: IO ()
main = do
  x <- readLn :: IO Double
  print (nilai x)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>ghc tantangan3.hs
[1 of 2] Compiling Main             ( tantangan3.hs, tantangan3.o )
[2 of 2] Linking tantangan3.exe

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>tantangan3.exe
65
"Grade B dan Skala 3"

C:\file kuliah ;p\{ifs25038} - pemfung-praktikum-3>
```